

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION High Performance Computing and
Big Data Analytics

DISSERTATION THESIS

Agent-Based Modelling of Brown Bear Population Dynamics in the Romanian Carpathians

Supervisor
Dr. NICULESCU Virginia, Conferentiar Universitar

Author
Sofian Razvan

2026

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Calcul de înaltă performanță și
analiza volumelor mari de date

LUCRARE DE DISERTAȚIE

Modelare bazată pe agenți a dinamicii
populației de urs brun din Carpații
României

Conducător științific
Dr. NICULESCU Virginia, Conferentiar Universitar

Absolvent
Sofian Razvan

ABSTRACT

Brown bear population dynamics in Romania represent both an ecological and a public-policy challenge. This thesis develops a spatial agent-based simulation of the brown bear population across Romania's forests and uses it to investigate the population increase observed in the 2005–2021 interval. The objective is explanatory and counterfactual: to compare plausible driver combinations under explicit assumptions, not to produce exact deterministic forecasts.

The model represents each bear as an autonomous agent with age, sex, satiety, reproduction, and movement behaviour, while the environment is represented as a heterogeneous grid with food and danger dynamics. The simulation architecture separates a parallel planning phase from a deterministic commit phase, and is supported by a reproducible experiment pipeline including seeded runs, schedule-driven parameter updates, and per-tick metrics recording.

The thesis combines ecological validation and computational analysis. Ecological results are evaluated through baseline and counterfactual scenarios, using trajectory-fit and population-structure indicators. In the final 2005–2021 campaign, the best fit is obtained by the anthropogenic-food schedule (MAE 261.25, RMSE 336.27), while baseline and policy-only schedules remain substantially less accurate. On the computational side, the model is executed with a multithreaded, single-machine design that parallelizes the per-agent planning phase, and built-in timing instrumentation is used to analyse where each tick spends its time. A distributed execution was deliberately not implemented: the analysis shows that for this fine-grained, tick-synchronized model the per-agent work is too small and the per-tick synchronization too frequent for distributing a single run to be worthwhile, because communication and synchronization overhead would dominate the useful work. The only level at which distribution would make sense is a batch of fully independent runs, but at the scale of these experiments a single multi-core machine was sufficient, so distribution was not needed.

Overall, the contribution is twofold: a practical ABM framework for hypothesis testing in brown bear population dynamics, and a reasoned, instrumentation-backed execution strategy that aligns model granularity with computational design.

Contents

1	Introduction	1
1.1	Context and motivation	1
1.2	Research problem and objectives	2
1.3	Scope and delimitations	3
1.4	Original contributions	3
1.5	Thesis structure	4
1.6	Use of generative AI tools	4
2	Background and related work	6
2.1	Brown bear ecology and demographic context	6
2.2	Simulation and the agent-based paradigm	7
2.3	Romanian case study framing	8
2.4	Demography and the problem of parameter realism	8
2.5	Methodological references for evaluation	9
2.6	Related software approaches	9
2.7	Gap addressed by this thesis	10
2.8	Chapter summary	11
3	Proposed approach	12
3.1	Model overview	12
3.2	Problem formulation and modelling assumptions	13
3.3	Agent state and behaviour	14
3.3.1	The decision pipeline	14
3.3.2	Movement as a weighted trade-off	15
3.3.3	Subadult male dispersal	16
3.3.4	Seasonality through hibernation	16
3.3.5	Reproduction and birth	17
3.3.6	Avoiding artificial birth waves at founding	17
3.4	Environment representation	18
3.5	Module-level architecture and the action-effects pattern	19
3.6	Execution architecture and concurrency	20

3.7	Reproducibility pipeline	20
3.8	Schedule-driven scenario modelling	21
3.9	Skipping the initialization transient	22
3.10	Performance instrumentation	22
3.11	Design limitations and current constraints	23
3.12	Multithreading, and why distribution was not applied	24
3.13	Chapter summary	25
4	Evaluation and validation	26
4.1	Evaluation strategy	26
4.2	Experimental protocol and scenarios	27
4.3	Ecological results	28
4.3.1	Quantitative summary	28
4.3.2	Where the trajectories diverge	29
4.3.3	Mortality structure	30
4.3.4	End-of-window interpretation	30
4.4	Trajectory and structure figures	31
4.5	Computational analysis	34
4.5.1	Where a tick spends its time	34
4.5.2	Why a single run should not be distributed	35
4.5.3	Where distribution would have belonged, and why it was still not needed	35
4.6	Interpretation and validity limits	36
5	Discussion, conclusion and future work	37
5.1	Discussion	37
5.2	Conclusion	38
5.3	Limitations	39
5.4	Future work	40
5.5	Final remarks	41
	Bibliography	42

Chapter 1

Introduction

1.1 Context and motivation

Over the last years, the brown bear population in Romania has quietly become one of the most visible and most argued-about wildlife-management topics in the whole of Europe. The debate is alive in the public media, in policy documents and in scientific reports at the same time, and the two sides of it pull in opposite directions. On one side there is a very real concern about safety and about the conflicts that happen at the edges of settlements, where bears and people meet more and more often. On the other side there is the scientific need to explain the long-term dynamics of the population using measurable mechanisms, rather than reacting to isolated incidents or to whichever number was published most recently. Romania holds the largest brown bear population in the European Union, so whatever happens here is not a local curiosity; it sets the tone for how a recovering large carnivore is managed across the continent [CKL⁺14, PBC⁺18].

The difficulty, and the thing that makes the question genuinely hard, is that many mechanisms act at the same moment. Hunting-policy changes, supplemental and anthropogenic feeding, the food that bears find near villages, the slow transformation of the landscape, and the strong seasonality of the animal itself all overlap and interact. If each of these is analysed on its own, the interpretation almost always comes out incomplete, because the interesting behaviour lives in how they combine. This is exactly the situation in which simulation becomes useful, not as a gadget but as a method: it lets us run controlled experiments in which each factor can be switched on, scheduled in time, and measured, and in which the same experiment can be repeated many times to see whether a result is robust or just luck.

This thesis proposes a spatial agent-based simulation for brown bears, inspired by the Romanian Carpathian context and implemented from the ground up in Java. The aim throughout has been to keep the biological meaning of the model intact

while keeping it computationally practical enough to run the large batches of experiments that a serious comparison actually requires. Those two goals are in constant tension, and a good part of the work described here is about managing that tension sensibly.

1.2 Research problem and objectives

The core research question can be stated in a single sentence: *which mechanisms, alone or in combination, can best explain the observed increase of the brown bear population in the Romanian Carpathian area during 2005–2021?* Everything in the thesis is organised around answering that question in a way that is reproducible and that does not overclaim.

To get there, the thesis sets itself five concrete objectives:

1. to design an agent-based model in which each bear is an individual with age, sex, satiety, reproduction and movement behaviour, so that population dynamics emerge from individual decisions rather than being imposed by an equation;
2. to represent the habitat and its risks as a spatially heterogeneous grid of cells carrying food and danger that change over time;
3. to evaluate candidate drivers of the increase using a baseline together with a set of counterfactual scenarios;
4. to build a reproducible experiment pipeline based on controlled random seeds, per-tick metrics recording and time-indexed parameter schedules, so that every result is traceable;
5. to study the computational behaviour of the model, with particular attention to why multithreading is the right tool for accelerating a single run, and why distributing a single run was judged not to make sense for a model of this granularity.

The first four objectives belong to the ecological side of the work and the fifth to its engineering side, but as the thesis will argue, the two are tightly connected: the kind of model that answers the ecological question well is exactly the kind of model whose computational behaviour is interesting and a bit counter-intuitive.

1.3 Scope and delimitations

It is important to be explicit and a little disciplined about scope, because a model can always be asked to do more than it honestly can. The study covers the **brown bear population across Romania’s forests**, essentially the Carpathian range, where almost the entire Romanian population lives, over a **2005–2021 analysis window**. That window is chosen because it is where the observed national increase is documented and where the policy and habitat context can be tied to real sources.

The purpose of the model is **explanatory counterfactual analysis**, and explicitly not exact numerical forecasting. In other words, the simulation is used to compare how plausible combinations of drivers manage to reproduce the observed increasing trend under stated assumptions; it is not used to claim a precise number of bears for some future year. I have chosen this scope for three reasons. First, it matches the granularity of the model, which is built on local interactions, tick-based dynamics and scenario schedules, and which is naturally suited to comparison rather than point prediction. Second, it is scientifically more robust given the real uncertainty in census methods and in the exact timing of policy periods. Third, it aligns with the technical contribution, where the value comes from running many controlled scenario replicates, which is again a comparative activity. The reader should therefore read every conclusion in this thesis as comparative explanatory evidence across scenarios, and never as a deterministic prediction.

1.4 Original contributions

The contributions of this thesis are both ecological and technical, and they are most interesting exactly where the two meet.

First, on the ecological side, the contribution is the integration of several candidate drivers inside one simulation framework. Instead of discussing one factor at a time, as much of the existing debate does, the project evaluates both isolated and combined effects, which makes it possible to talk about relative contribution and about interaction between drivers in a disciplined way.

Second, on the technical side, the contribution is a reasoned execution strategy that is realistic for this particular class of model. The implementation is multi-threaded on a single machine: the per-agent planning phase is parallelised, while the world update stays serial. A distributed execution, on the other hand, was deliberately *not* built, and the thesis explains why rather than treating “distributed” as an automatic mark of seriousness. For a fine-grained, tick-synchronised simulation like this one, distributing a single run would introduce communication and synchronisation overhead that is larger than the tiny per-agent work it would par-

allelise, so it would most likely run slower, not faster. The only level at which distribution would help is a *batch of independent runs*, which is embarrassingly parallel, but at the scale of these experiments a single multi-core machine handled the whole campaign comfortably, so distribution was never necessary. Making this judgement explicitly, and backing it with the timing instrumentation, is one of the things that set this work apart from a model that simply runs and prints a graph.

Third, the project bakes reproducibility directly into the code rather than treating it as an afterthought: run configurations, seeded randomness with per-agent streams, per-tick metrics, schedule-based parameter updates and calibration utilities are all first-class components.

Fourth, and more modestly, the thesis contributes a clear scope statement that keeps the claims proportional to the evidence, explanatory counterfactual analysis over 2005–2021 for the Romanian brown bear population, which is itself a small act of scientific hygiene that ecological simulation work does not always practise.

1.5 Thesis structure

The thesis is organised in five chapters followed by the references. Chapter 2 sets out the ecological background and the related work in agent-based and parallel simulation, and it states precisely the gap that this project addresses. Chapter 3 describes the proposed approach in detail: the structure of the model, the decision logic of the agents, the representation of the environment, the action–commit execution architecture, and the reproducibility and scheduling machinery that turns the simulator into a scientific instrument. Chapter 4 presents the evaluation, in two halves, the ecological results of the 2005–2021 campaign and the computational findings that justify the execution strategy. Chapter 5 then discusses what the results mean, states the limitations honestly, draws the conclusions, and lays out the directions in which the work can be taken further.

1.6 Use of generative AI tools

For full transparency, I want to state clearly where generative AI tools were used in this work. They were used in a limited way, to help develop the user-interface (frontend) part of the application, that is the visual layer through which the simulation is displayed and controlled. Concretely, I used such tools as an assistant while writing the frontend code, asking for boilerplate, for help with layout and rendering details, and as a starting point that I then reviewed, adapted and integrated into the project. The scientific substance of the thesis, that is the agent-based model, its

mechanisms and parameters, the experiment pipeline, the scenarios, the results and their interpretation, is my own work.

Chapter 2

Background and related work

2.1 Brown bear ecology and demographic context

Brown bear ecology is a very good case for simulation, because the population trends we read about in reports are never the product of a single cause. They emerge from many local processes acting at the same time: reproduction, mortality at different ages, access to food, the way individuals use space, and the constant friction of living next to humans. In practice none of these processes acts alone. They overlap in time and they interact, and their combined effect is what pushes a population toward growth, decline, or a local equilibrium. This is precisely why a verbal, one-cause-at-a-time explanation tends to fall short, and why a model that lets the causes act together is useful.

From the demographic literature we know that brown bear trajectories are shaped above all by age and sex structure, by the long intervals between successive litters, by cub survival, and by the mortality pressure that human-dominated landscapes add on top of natural mortality [SMH03, SES⁺98, SSS⁺97]. These are not abstract details for a modeller; they are the realistic bounds that keep the model honest. When the simulation in this thesis uses a minimum reproductive age of four years, a gestation of about a year, a multi-year cooldown between litters, a litter mean close to two cubs, and elevated mortality for cubs, none of those numbers is invented. They come from this body of work, and the role of the literature is exactly to provide defensible ranges instead of arbitrary values [SGDZ00].

The Romanian context makes the problem more relevant still. Romania hosts by a wide margin the largest brown bear population in the European Union, and recent work has documented both the movement ecology of these bears and the patterns of human–bear interaction in the Carpathians [PBC⁺18, PIP⁺17]. At the same time, institutional reports and policy documents point to substantial changes both in the estimated size of the population and in the management strategy applied to it over

the last two decades [INC23, Min24]. The increase in conflict reports and the shifting estimates of population size are not just background colour; they are the very signal this thesis tries to explain. So this is not only a theoretical modelling exercise. It is a problem with direct public-policy weight, where the question of which mechanism actually drove the rise has practical consequences for how the population should be managed [CKL⁺14].

2.2 Simulation and the agent-based paradigm

Agent-based modelling is the natural tool to reach for when individual heterogeneity matters and when interactions are local in space and time, and both of those are true for bears. Instead of forcing the whole population into one aggregated equation, an agent-based model lets each individual carry its own internal state and update its behaviour according to its own local context. The population-level behaviour is then not assumed in advance; it appears as an emergent consequence of all those individual updates. For a question like ours, where we want to test whether a given assumption is *sufficient* to produce an observed trend, this emergence is the whole value of the method.

The methodological foundations of individual-based ecological models are by now well established [GR05, RG19, DM05]. Just as importantly, the field has developed shared standards for describing such models, most notably the ODD protocol and its update, which lay out a practical structure for stating a model's entities, processes, scheduling and reproducibility decisions [GBB⁺06, GRV⁺20]. This thesis follows the same spirit, even where it does not follow the protocol to the letter: Chapter 3 explicitly defines the agent state, the scheduling of the tick loop, the environment representation and the experiment protocol, precisely so that the model can be understood and, in principle, reproduced.

The reason this paradigm fits counterfactual analysis so well is that interventions can be expressed as explicit rules and then switched on and off. If we want to test the effect of changing hunting pressure or of adding a food subsidy, we do not have to re-derive an equation. We change a rule or a parameter at a given year and re-run, and because the model is stochastic, we re-run it several times and look at the distribution of outcomes rather than a single line. This is the same logic used across the wider agent-based modelling community, from general treatments of the method [Mac16] to teaching-oriented platforms such as NetLogo [WR15].

2.3 Romanian case study framing

In this thesis the Romanian situation is treated as a concrete testbed, not as a generic backdrop. More precisely, the empirical framing is the brown bear population across Romania's Carpathian forests as a whole, with 2005–2021 as the main analysis window. Choosing a real framing rather than a purely abstract one brings two clear advantages.

First, it provides real-world constraints. Demographic assumptions, habitat structure and the policy timeline can all be tied to published or institutional sources, which sharply reduces the number of arbitrary modelling choices and makes the scenarios defensible. The landscape transitions of the Romanian Carpathians over recent decades have themselves been studied [MKB⁺14, GMKH13], which gives some grounding to the habitat-loss scenario we test later. Second, a real framing allows the results to be interpreted in a management-oriented direction. Even though the model stays simplified, its scenario outputs can still be read as statements like “this combination of pressures reproduces the observed trend more plausibly than that one”, which is the kind of statement a manager can actually use.

At the same time, this framing brings a responsibility in how the results are read. Institutional population estimates evolve as census methods improve, and policy periods do not always begin and end on clean calendar boundaries. For that reason the simulation is presented as a structured comparative tool under explicit assumptions, and not as an exact forecasting device. I would rather make a careful comparative claim that holds than a precise numerical claim that the data cannot support.

2.4 Demography and the problem of parameter realism

There is a recurring weakness in student simulation projects that I was determined to avoid, and it is worth naming directly: parameter arbitrariness. In ecological agent-based modelling it is dangerously easy to produce plots that look plausible but that mean very little, because the parameters behind them were tuned by hand until the picture looked nice. A model like that can be made to show almost anything, which means it shows nothing.

The defence against this is to constrain parameter ranges with the demographic literature wherever possible, and only then to calibrate within those ranges. The references used here support realistic intervals for the quantities that matter most: age-related mortality, the vulnerability and survival bottleneck of cubs, the intervals between litters, and the additional mortality that human pressure imposes [SMH03, SES⁺98, McL89, BSY⁺09]. The sex- and age-specific mortality multipliers

in the model, the elevated cub danger, and the long reproductive cooldown all sit inside ranges drawn from this work. This does not make the uncertainty disappear, but it does give a defensible starting point for the calibration and sensitivity experiments, which is a very different situation from picking numbers until the curve fits.

2.5 Methodological references for evaluation

Besides the ecological literature, the thesis also leans on methodological sources, because how you evaluate a stochastic simulation is itself a methodological question. Sensitivity analysis and parameter-estimation workflows for this kind of model are well covered in the standard references [SRA⁺08, TKG14], and they inform the way Chapter 4 reports its results. The decision that follows from them is simple but non-negotiable: if a claim depends on a stochastic simulation, a single run is not evidence. Every scenario in this thesis is therefore run as several seeded replicates and then aggregated, and the comparison against the reference trajectory is made on those aggregates rather than on one lucky or unlucky line.

2.6 Related software approaches

From the software side there is a mature body of prior work on multi-agent simulation environments, including MASON and Repast and their high-performance, distributed variants [LCRP⁺05, CN13, NCO⁺13], lighter frameworks such as Mesa [MK15], and dedicated spatially explicit ecological platforms [BPP⁺14]. There is also a long-standing theoretical foundation for parallel and distributed simulation in general [Fuj00]. I chose to build the model directly in Java rather than on top of one of these frameworks, partly for full control over the execution and reproducibility machinery, and partly because the computational question I wanted to ask is about that very machinery.

What this literature makes clear, and what is central to my argument, is that the benefit of parallel or distributed execution depends entirely on the shape of the workload: on how heavy the per-agent computation is, on the communication pattern, and on how often the model has to synchronise. In models where each agent does heavy local computation between synchronisations, distribution scales well. In models where the per-agent work is light and synchronisation is frequent, communication overhead can erode the speedup and even reverse it. This distinction is exactly the one that applies to the bear model. Because it is tick-based and globally synchronised, and because each agent action is cheap, distributing a single run can

be slower than running it multithreaded on one machine, while distributing many independent runs remains very effective. The wider ecological agent-based literature, including work on other large carnivores such as tigers [CLBG15], confirms more generally that complex wildlife dynamics can be represented faithfully at the individual level and produce meaningful scenario outputs, which is the methodological reassurance this thesis needed before committing to the approach.

2.7 Gap addressed by this thesis

The gap this thesis addresses has two sides, one ecological and one computational, and the contribution is to close both at once with a single piece of work.

On the ecological side, much of the public and even technical discussion about bear-population change in Romania stays descriptive, and it tends to emphasise one driver at a time, it was the hunting ban, or it was the feeding, or it was habitat change, without ever testing how these drivers behave together inside one consistent framework. This thesis addresses that by integrating several candidate drivers into one model and evaluating them, both in isolation and combined, under controlled scenarios with replicated runs.

On the computational side, reports and student projects alike often present multithreaded and distributed execution as if they were universally beneficial, almost as a synonym for “more serious”. In practice, as the simulation-systems literature warns, that is not true for every workload. For this tick-synchronised model the thesis works out explicitly where parallelism actually helps, and, just as importantly, where it would not. The model is therefore executed with a multithreaded, single-machine design, and a distributed execution is deliberately *not* built, because the analysis shows it would not pay off for this granularity. The reasoned conclusion is a simple two-level picture:

- multithreading is used to accelerate a single simulation run on one machine, where the threads share memory and the per-tick synchronisation barrier is cheap;
- distribution would only make sense at the level of large batches of independent runs, which are embarrassingly parallel, but at the scale of these experiments a single multi-core machine ran the whole campaign comfortably, so it was not needed and not implemented.

Stating this judgement explicitly, rather than reflexively reaching for a distributed implementation, improves both the scientific reproducibility of the results and the engineering honesty of how they were produced, and it is one of the things that I believe distinguishes this work from a model that simply runs and prints a curve.

2.8 Chapter summary

This chapter set out the conceptual and bibliographic foundation of the thesis. The ecological context motivates the modelling problem and supplies realistic parameter ranges; the agent-based literature supports the choice of paradigm and the standards for describing and evaluating such a model; and the simulation-systems literature supplies the criteria for the computational part of the evaluation. With this foundation in place, the next chapter moves from background to implementation: how the world is represented, how agents decide and act, how their effects are committed each tick, and how the whole thing is wrapped in a reproducible experiment pipeline.

Chapter 3

Proposed approach

3.1 Model overview

The proposed solution is a spatial agent-based simulation written in Java, in which every brown bear in the studied area is represented as one individual software agent that lives, eats, moves, reproduces and eventually dies inside a shared, heterogeneous world. The choice of an agent-based model is not accidental. The phenomenon we are trying to understand, namely the increase of the Romanian brown bear population during 2005–2021, is the cumulative result of a very large number of small, local decisions: a single bear deciding whether to look for food, a single female deciding whether the conditions are good enough to reproduce, a single subadult male leaving the area where it was born. None of these decisions is interesting on its own, but together, repeated over many thousands of individuals and many simulated years, they produce the population trend that we actually observe in the field. The whole point of building the model this way is that the population trajectory is never written down anywhere in the code. It is an *emergent* property, and this is exactly what we want, because it lets us ask whether a given combination of assumptions is able to reproduce the real trend without us forcing the answer by hand.

Time in the simulation is discrete and advances in equal ticks. One tick corresponds to a fixed fraction of a simulated year, and in the current calibration one tick is set to 0.0001140771 years, which is approximately one hour. This means a full simulated year is built out of roughly 8766 ticks, and the 2005–2021 window of sixteen years is therefore around 140,000 ticks long. Working at an hourly resolution may look excessive for a population-level question, but it is a deliberate decision: it lets behaviours such as feeding, daily movement and the slow decay of body condition play out at a believable pace, instead of being collapsed into one coarse yearly update where everything happens at once.

At each tick, the model follows a two-stage flow, which is one of the most important design decisions of the whole project:

1. **Plan stage:** every agent looks at its own local surroundings, receives a percept, and decides on exactly one action for this tick. No agent is allowed to change the world during this stage. It only decides.
2. **Commit stage:** all the decided actions are collected and then applied to the shared world in one controlled, ordered pass on a single thread.

This separation between deciding and acting is what makes the rest of the architecture possible. Because nobody modifies the world while everybody is still thinking, the planning stage is free of data races and can be run in parallel across many threads without any locking on the world state. And because all the modifications are funnelled through one ordered commit, the resulting world transition is deterministic and easy to reason about. We will come back to this idea several times in this chapter, because almost every other design choice depends on it.

3.2 Problem formulation and modelling assumptions

Before describing the implementation in detail, it is worth stating clearly what kind of model this is and what it is not. The modelling objective is explanatory and counterfactual: we want to compare plausible explanations for the observed population increase under controlled, repeatable computational experiments. We are not trying to produce an exact deterministic forecast of how many bears Romania will have in a given future year, because the available census and policy data simply do not support that level of precision. What the model *can* do well is answer a comparative question, of the form “under these assumptions, does this driver, alone or combined with others, reproduce the observed trend better than that one?”.

In order to keep the model both interpretable and actually executable for large batches of runs, several assumptions are introduced, and it is worth listing them so the reader knows where the simplifications are:

- the world is a regular two-dimensional grid, and interactions are local, restricted to a small neighbourhood around each agent;
- time advances in equal discrete ticks, with no continuous-time events between ticks;
- each agent perceives only its immediate surroundings, never the global state of the population;

- mortality and reproduction are probabilistic, but the probabilities are bounded by biological rules (minimum reproductive age, gestation, cooldown, litter limits);
- the environment is abstracted into food and danger fields layered on top of a small set of terrain categories.

None of these assumptions claims perfect realism, and I do not want to pretend otherwise. Their role is to strike a balance between biological plausibility on one side and reproducibility and computational tractability on the other. A model that is too detailed becomes impossible to calibrate and impossible to run thousands of times; a model that is too coarse stops being biologically meaningful. The assumptions above are the compromise that this thesis settled on, and the parameter values behind them are, wherever possible, tied to the demographic literature rather than chosen arbitrarily [SGDZ00, SMH03].

3.3 Agent state and behaviour

Each bear agent is implemented in the *BearAgent* class and carries a compact internal state. The fields that matter for behaviour are the bear's identity, its sex, its age in years, its satiety, its reproduction cooldown, its gestation progress, whether it is currently pregnant, and a remembered home-range anchor expressed as a pair of coordinates. It is important to notice that the agent does not store its own position on the map. The position lives in the world state, not in the agent, which keeps a clean separation between "who the bear is" and "where the bear is", and turns out to be very convenient when we save and restore whole populations later.

3.3.1 The decision pipeline

The heart of the agent is the method that decides the next action, and it is best described as a short pipeline that is evaluated in a fixed order at every tick. The order is meaningful, because the first condition that matches wins and the rest are skipped.

The very first thing that happens each tick is the passage of time. The agent ages by one tick, its reproduction cooldown and remaining gestation are both decremented, and its satiety decays. Only after time has passed do we check whether the bear has died. Death from starvation comes first: if satiety has reached zero, the agent decides to die with cause `STARVATION`. Next comes old age: once a bear is older than the configured maximum age, every tick it rolls against a small per-tick death probability, scaled by a sex-specific multiplier because in the model, as in reality, females are slightly longer-lived than males. After that comes danger: the dan-

ger value of the cell the bear currently stands on is turned into a per-tick probability of dying, and this probability is multiplied by several factors: a large multiplier for cubs, who are far more vulnerable, a sex multiplier because roaming males are more exposed, and a strong reduction during hibernation because a dened bear is sheltered.

If the bear survives all three mortality checks, the pipeline turns to reproduction and behaviour. A pregnant female whose gestation has finished gives birth, resets her cooldown and clears her pregnancy. If the bear is hibernating, the pipeline stops here: a dened bear does not forage, does not move and does not mate, although births can still happen in the den. Outside hibernation, an eligible female that perceives a male nearby becomes pregnant and starts her gestation clock. Then comes feeding: a hungry bear, meaning one whose satiety is below an eat-preference threshold, eats immediately if there is enough food on its current tile. If the bear is not hungry enough to eat in place, it evaluates its neighbouring cells and moves to the best one, provided that cell is actually better than staying put. And if there is no better neighbour but there is food where it stands, it eats anyway; otherwise it simply does nothing and waits for the next tick.

It is worth mentioning that this ordering is itself a modelling decision. By putting survival before reproduction and reproduction before feeding and movement, the agent expresses a simple priority: stay alive, then continue the species, then take care of daily needs. It is a deliberately reactive design rather than a planning one, because for a population-level question we care about the statistical behaviour of many bears, not about any single bear executing a clever long-term plan.

3.3.2 Movement as a weighted trade-off

Movement deserves its own discussion, because it is where most of the spatial realism of the model comes from. When a bear is not eating in place, it looks at the eight cells around it (a Moore neighbourhood) and gives each candidate cell a score. The cell with the highest score wins, and the bear moves there unless its current cell already scores best. The score is a weighted combination of four ingredients: the food on the cell pulls the bear in, the danger on the cell pushes it away, the local crowding (how many other bears are nearby) pushes it away, and a home-range penalty discourages the bear from wandering too far from the anchor point it remembers. A small amount of random noise is also added, so that bears do not behave like perfectly identical optimisers and the movement looks more natural.

The really interesting part is that the weights are not constant: they shift depending on how hungry the bear is. A well-fed bear weighs danger heavily and food lightly, so it behaves cautiously and stays away from risky cells such as villages and

roads. A hungry bear does the opposite: the food weight grows, the danger weight shrinks, and the crowding and home-range weights are reduced as well. In plain words, a hungry bear is willing to take risks and to leave its comfortable range to find something to eat, which is exactly the kind of behaviour that pushes bears toward human settlements when natural food is scarce, and which is at the centre of the anthropogenic-food hypothesis we test in the next chapter. The hunger level is computed as a simple normalisation of satiety, and the weights are linearly interpolated between a “when full” value and a “when hungry” value, so the transition is smooth rather than a hard switch.

3.3.3 Subadult male dispersal

On top of the generic movement logic, the model adds a specific behaviour for subadult males, because in brown bears natal dispersal is overwhelmingly a male, subadult phenomenon [McL89, SEH⁺12]. A male bear inside a configured age window (roughly two to six years old) is treated as a disperser: its crowding weight is multiplied up so it actively avoids competition, its home-range weight is multiplied down so the anchor barely holds it, and it receives a positive bonus for every cell it places between itself and its birth range. The combined effect is that young males drift outward from where they were born, which spreads the population across the map instead of letting it pile up around the founding locations. This matters for the realism of the spatial pattern and, indirectly, for how the population finds and uses new food.

3.3.4 Seasonality through hibernation

Brown bears are strongly seasonal animals, and ignoring this would make the yearly dynamics wrong. The model represents seasonality with a hibernation window defined as a fraction of the year, configured to run from roughly early November to early March, wrapping around the year boundary. While a bear is inside this window, two things change. First, its metabolism slows down: satiety decays much more slowly, controlled by a hibernation metabolism multiplier well below one, so a dened bear can survive the winter on its reserves. Second, its exposure to danger drops sharply, because a sheltered, denning bear is far less likely to encounter the risks of the active season. During hibernation the agent does no foraging, no movement and no mating, but, importantly, births still occur, since brown bear cubs are in fact born in the den during winter. This single mechanism is responsible for a lot of the believable seasonal texture in the output.

3.3.5 Reproduction and birth

Reproduction in the model is gated by several biological constraints that all have to be satisfied at once. A female can only conceive if she is not already pregnant, is older than the minimum reproductive age, has satiety above a minimum threshold, is off her reproductive cooldown, and perceives a male nearby. The satiety requirement is what couples reproduction to food: in a poor environment, fewer females clear the bar and the birth rate falls on its own, without us having to script it. Once pregnant, the female carries for a gestation period of about one year, and when it elapses she gives birth and enters a multi-year cooldown before she can reproduce again, which reproduces the long inter-birth intervals that are characteristic of the species.

The litter is not a fixed number. Its size is drawn from a normal distribution around a calibrated mean (close to two cubs), clamped to be at least one. Each cub is then subjected to an infant-mortality probability at birth, representing perinatal losses, so not every cub in the litter is actually added to the live population. This two-step birth process, variable litter size followed by per-cub survival, gives a more realistic recruitment signal than simply adding a fixed number of healthy cubs per mother.

3.3.6 Avoiding artificial birth waves at founding

There is a subtle but important detail in how the founding population is created. If every founding female of reproductive age started the simulation in exactly the same reproductive state, they would all come off cooldown or finish gestation at the same time, producing an artificial wave of synchronised births that has nothing to do with ecology and everything to do with how we set up the run. To avoid this, the founding reproductive females are staggered: a fraction of them start already pregnant with a random remaining gestation, and the rest start with a random remaining cooldown. The founding ages themselves are not drawn uniformly either. A flat age spread from zero to twenty-five years would have a mean around twelve and a half, which is roughly double that of a real brown bear population and over-represents middle-aged bears, again triggering a boom-then-crash transient. Instead, founders are sampled from a young-skewed, truncated-exponential age pyramid with a mean of about seven years, which is much closer to a real stable age distribution [SSS⁺97, GR05]. These two choices together make the population start near its stationary structure rather than lurching into it.

3.4 Environment representation

The world is a square grid, and each cell carries a terrain type together with a food value and a danger value, both living in the interval $[0, 1]$, plus explicit occupancy information. The terrain types are forest, field, village, road, mountain and a blocked boundary type. When a random map is generated, the terrain mix is controlled by configurable percentages: in the current settings about a quarter of the map is forest, more than half is field, a tenth is mountain, and small fractions are road and village. Each terrain type then has its own average food and average danger, with a bounded random variance on top so that no two cells of the same type are identical. Forest is the rich, safe habitat (high average food, low danger), villages are food-rich but extremely dangerous, roads are poor and dangerous, and fields and mountains sit in between. This is a compact way of encoding the central ecological tension of the whole problem: the safest food is in the forest, but the easiest, most concentrated food is exactly where it is most dangerous to be, near people.

Food is not static. Bears reduce the food on a cell when they eat there, and the environment regrows food over time, restoring a random number of cells each tick. The amount eaten per tick and the amount regrown per tick are both parameters, and the ratio between them effectively sets the carrying capacity of the landscape. This is precisely the lever that the anthropogenic-food scenario pulls: increasing the regrowth rate over the years is how we represent more food becoming available to bears, whether through supplemental feeding, crops, or waste near settlements.

The environment also exposes a percept for each agent. The percept contains the bear's current cell with its food, danger and local bear count, the eight neighbouring cells with the same information plus a blocked flag, and a single boolean saying whether a male is nearby, which the females use for the reproduction check. The perception is strictly local; an agent never sees beyond its immediate ring of cells, which keeps the model faithful to the idea that bears act on local information.

Finally, the implementation can either generate a random map from the terrain percentages or load a pre-processed map grid from a file. This second capability is what allows the project to move beyond purely synthetic landscapes toward geographically structured experiments, where the grid reflects a real habitat layout rather than a statistical mix.

3.5 Module-level architecture and the action–effects pattern

To keep the implementation understandable and to let the ecological logic, the execution policy and the experiment workflow evolve independently, the code is separated into modules with clear responsibilities. The *BearAgent* module holds the internal state and the decision logic. The *BearEnvironment* module builds percepts and applies state transitions. The *BearSimulation* driver owns the tick loop, the phase boundaries and the scheduling. A dedicated effects container decouples deciding from mutating. And an *Experiments* module handles run configuration, metrics output, schedules and calibration tools. This is the same separation-of-concerns principle that runs through the whole thesis: keep things that change for different reasons in different places.

The action–effects pattern is worth describing on its own because it is what makes the parallel-plan / serial-commit idea actually work. When an agent decides on an action, the action is not applied directly to the world. Instead, each action object, when asked to contribute to the current step, writes its *intent* into a shared per-tick *BearActionEffects* container. A move writes a move intent keyed by agent id; a death writes a die intent together with its cause; a birth writes a birth intent naming the mother; eating writes a food-consumption intent for a cell; and a counter is kept for human-conflict exposure. Nothing in the world has changed yet at this point, we have only gathered a description of what everybody wants to do.

The commit phase then resolves these intents in a fixed, deliberate order on the driver thread: first deaths are applied and the dead bears are removed, then births add new cubs after litter sampling and infant mortality, then food reductions are applied to the eaten cells, then move intents are resolved, then conflict exposure is recorded, and finally food is partially regrown. Applying intents in a fixed order is what gives the model deterministic semantics: there is never a question of which of two simultaneous changes “won”, because the order is always the same. A pleasant side effect of centralising every change through this container is observability, because all the intents pass through one place, it is trivial to count births, deaths by cause, conflict events and so on for each tick and export them, which is the backbone of the entire results chapter.

One detail about movement is worth being honest about: the model does not enforce that a cell can hold only one bear. Move intents are grouped by their target cell and every claimant is moved there. Bears can therefore share a cell, and competition between them is expressed softly, through the crowding penalty in the movement score and through the shared food on the cell, rather than through a hard collision rule. This is a simplification, but it is a reasonable one at the scale of the question

we are asking.

3.6 Execution architecture and concurrency

The simulation uses parallel planning and serial commit as its core execution strategy, and the implementation leans on a modern Java feature to do it cheaply. During planning, one small task is created per agent. Each task fetches that agent's percept and asks the agent to decide its action, and all of these tasks are submitted to an executor that uses *virtual threads* (`newVirtualThreadPerTaskExecutor`). Virtual threads are extremely lightweight, so we can afford to create one task per bear even when there are several thousand bears, without worrying about exhausting operating-system threads. The driver then collects the results into an ordered map keyed by agent id and hands them to the commit phase.

The reason this works without any locks on the world is precisely the plan/commit separation discussed earlier. During planning, the workers only *read* the world to build percepts and they only *write* to the agent's own private state, never to shared structures. The shared world is touched only in the commit phase, which runs on a single thread. From a software-engineering point of view, this design gives three concrete advantages: safe concurrency for the expensive decision-making, a consistent and reproducible world transition per tick, and a single natural place to plug in metrics, schedule hooks and reproducibility controls.

There is a trade-off here, of course: the serial commit is a hard synchronisation barrier. Every tick, all the parallel work has to finish and then funnel through one thread, and the loop cannot advance until the commit is done. At very large scales the commit can become the bottleneck, and, as we will argue in the computational discussion, this same barrier is exactly what makes distributing a *single* run across machines unattractive. In the current version this trade-off is accepted on purpose, because correctness and interpretability are prioritised over squeezing out the last bit of low-level performance.

3.7 Reproducibility pipeline

Probably the strongest part of the whole project is the reproducibility infrastructure, due to the fact that it is what turns the simulator from a nice demonstration into something that can actually back up a scientific claim. The principle behind it is simple: every reported number must be traceable back to a scenario definition and a random seed.

The trickiest part of reproducibility in a parallel simulation is randomness. If all

agents drew from one shared random generator, then the order in which the threads happened to run would change the sequence of draws, and the same seed would give different results on different machines or even different runs. The *RngSupport* module solves this with per-agent random streams. In seeded mode, each agent gets its own *Random* instance whose seed is derived from the master seed and the agent id through a SplitMix64-style mixing function. Because the stream depends only on the pair (master seed, agent id), a given agent always sees exactly the same sequence of random numbers regardless of which worker thread happens to execute its decision. A separate environment-level generator, used only from serial code such as map generation and the commit phase, handles all the world-level randomness. The per-agent generators are kept in a concurrent map so that threads planning different agents never contend when a new agent's stream is created. When no seed is set, everything falls back to *ThreadLocalRandom*, so ordinary interactive runs pay no penalty for machinery they do not need.

Around this sits the rest of the pipeline. A run configuration object captures the scenario id, the run id, the seed, the replicate index and the maximum number of ticks. A per-tick metrics recorder samples the population, births, deaths by cause, satiety, age structure and conflict events, and the founding cohort is recorded before any tick runs so that the initial gender split and founding-pregnancy count are visible too. The *BatchRunner* ties it together: it runs several seeded replicates of each scenario, applies any parameter overrides to the global settings before a run and restores them afterwards, writes one per-tick CSV per replicate, and writes a metadata CSV and a summary CSV with the fit metrics. Every result in the next chapter comes out of this pipeline.

3.8 Schedule-driven scenario modelling

A static model cannot represent the question we are asking, because the phenomenon itself is time-dependent: protection policies came into force in particular years, hunting rules changed, food availability shifted over the period. The model therefore supports time-indexed parameter changes through a *ParameterSchedule* and a *ScheduleApplier*. A schedule is simply a list of entries, each saying “at this simulation year, set this parameter to this value”. As the simulation clock crosses each scheduled year, the applier mutates the corresponding field.

The implementation does this through reflection on the static *Settings* class, which is an elegant point I am quite happy with: a schedule can target *any* settings field by name, without the simulation code having to know in advance which parameters a scenario will want to change. The applier looks up the named field, checks that it is static, parses the new value to the field's type, and sets it. Crucially, the first

time it touches a field it snapshots the original value, and it records every change it makes as an applied event with the tick, the year, the parameter, the old value and the new value. At the end of the run it restores every field it ever touched, so one run can never silently contaminate the next, and the recorded events become a per-run audit trail that is written out alongside the results. This is how scenarios such as “danger mortality drops from year two” (the 2007 EU protection driver), “food regrowth increases stepwise in years five, ten and fifteen” (anthropogenic food), or a layered combination of several such changes are expressed, all without writing any scenario-specific code in the simulation loop itself.

3.9 Skipping the initialization transient

A practical problem with starting every run from freshly generated random founders is the initialization transient: even with the young-skewed age pyramid and the staggered reproductive states described earlier, the artificial founding population needs a few simulated years to settle into a realistic spatial and demographic structure, and during that settling the trajectory is not yet meaningful. Running every scenario through that warm-up over and over is wasteful and, worse, it injects a wobble into the early years that we then have to explain away.

The project addresses this in two complementary ways. The first is snapshotting: the simulation can be run once until the population has spun up, then a *BearSnapshot* of the whole world and population is written to disk, and later runs can restore that snapshot instead of generating founders, so they start from an already-settled state with the clock reset to zero. The second is the *BalancedInitializer*, which goes a step further: it reads a characterization of the measured stable state (age distribution, satiety, sex, reproductive state, per-habitat placement) and generates a fresh founding population directly from those distributions, while also initialising the map’s food to its drawn-down level rather than a full larder. Both approaches remove the transient; the difference is that the snapshot restores one specific frozen population exactly, while the balanced initializer samples a new but statistically equivalent one. For the campaign in the next chapter this matters a great deal, because it means the scenarios all start from the same realistic baseline and the differences we measure between them are due to the drivers, not to leftover warm-up noise.

3.10 Performance instrumentation

So that the computational discussion can be based on evidence rather than impression, the simulation contains a detailed timing instrumentation, active when bench-

marking is enabled. The loop time is split into the planning time and the commit time, and each of those is decomposed further. On the planning side we record the time spent building the worker tasks, invoking them all, and collecting their futures, plus the aggregate worker CPU time spent on percept generation and on decision-making (measured per worker and therefore non-additive, since the workers run concurrently). On the commit side we record, separately, the time spent contributing actions into the effects container, applying deaths, applying births, applying food reductions, resolving moves, regrowing food and recording performed actions. There is also an explicit accounting of “unaccounted” overhead in each phase, so the numbers add up honestly.

This level of detail is what lets us say something precise instead of vague in the next chapter. Rather than claiming that one execution mode is “faster”, we can point to exactly where the time goes, and, in particular, show how much of each tick is spent in the serial commit barrier, which is the quantity that decides whether distributing a single run could ever pay off.

3.11 Design limitations and current constraints

It is worth being honest about the current limits of the architecture, and these are best understood as the boundary of where the approach currently applies rather than as defects.

First, many parameters live in a single static *Settings* class. This is convenient and it is what makes the reflection-based scheduler so simple, but global mutable state has to be handled carefully: the schedule applier’s snapshot-and-restore discipline exists precisely to stop one run leaking into another, and strict isolation between runs depends on that discipline being respected. Second, the tick barrier imposes strict phase boundaries; this is what guarantees determinism, but it is also what limits how far a single run can be distributed across machines. Third, several ecological processes are intentionally abstracted, food is a scalar field rather than distinct food types, conflict is a proxy count of bears on village and road cells rather than a modelled human response, and there is no genetic or individual-quality variation between bears beyond age and sex. These are conscious simplifications appropriate to the scope of this thesis and to the comparative question being asked, and each of them is a natural direction for future extension.

3.12 Multithreading, and why distribution was not applied

It is worth being completely clear about this, because it would be easy to overstate it: the model is executed in a multithreaded way on a single machine, and a distributed execution was *not* implemented anywhere in the project. This is a deliberate decision rather than an omission, and the reasoning follows directly from everything above. I want to explain why distribution simply did not make sense here, instead of bolting on a distributed mode just so the project could carry the label.

Each agent's per-tick work, that is building a local percept, scoring eight neighbouring cells and picking an action, is very small, on the order of microseconds. At the same time the model is tightly tick-synchronised: every tick ends in a serial commit barrier that all agents must reach before anyone can advance. If one tried to spread the agents of a single run across several machines, every single tick would require shipping each machine's intents back to a coordinator, resolving them, and shipping the updated world back out, all before the next tick could begin. The communication and synchronisation cost of doing that, tens of thousands of times over a sixteen-year run, would dwarf the tiny amount of useful work it parallelises, so a distributed single run would almost certainly be *slower* than the multithreaded one, not faster. This is a well-known pattern in the parallel and distributed simulation literature: fine-grained, frequently synchronised workloads do not distribute well, whereas coarse-grained ones do [Fuj00].

The right way to think about parallelism here is to match it to the granularity that actually exists in the problem:

- multithreading, with the lightweight virtual-thread planning stage, is what the project uses to accelerate the agent decisions *inside* one run on a single machine, where the threads share memory and the commit barrier costs essentially nothing;
- distribution would only ever belong at the level of *many independent runs*, different scenarios and different replicate seeds, since those runs share nothing and never synchronise with each other.

That batch level is the embarrassingly parallel part of the problem, and it is the only place distribution could have actually paid off. But in practice it was not needed: the whole experiment campaign is a manageable number of independent seeded runs, and a single multi-core machine ran them in acceptable time, so there was no reason to add the complexity and the failure modes of a distributed setup. The honest summary, then, is that the project is a multithreaded agent-based model

whose design was *analysed* for distribution and found not to warrant it, which is itself one of the more useful technical findings of the thesis, because it pushes back on the common assumption that more machines automatically means a faster simulation.

3.13 Chapter summary

This chapter described the proposed approach in full, from the modelling assumptions and the agent’s decision pipeline, through the environment and the action-effects commit logic, to the concurrency model, the reproducibility pipeline, the schedule-driven scenarios, the spin-up machinery and the performance instrumentation. The recurring theme has been the plan/commit separation and the decision to keep ecological logic, execution policy and experiment workflow apart from one another. The next chapter puts all of this to work: it defines the evaluation protocol and reports both the ecological results of the 2005–2021 campaign and the computational findings that justify the multithreaded single-machine execution and the reasoned decision not to distribute.

Chapter 4

Evaluation and validation

4.1 Evaluation strategy

The evaluation in this thesis has two distinct halves, and it is worth separating them clearly because they answer different questions. The first half is ecological: does the model, under a given set of assumptions, reproduce the observed brown bear population increase over 2005–2021, and which driver or combination of drivers reproduces it best? The second half is computational: does the execution strategy proposed in Chapter 3 actually hold up when measured, that is, is multithreading genuinely the right tool for a single run, and does the timing really justify the decision not to distribute? The two halves are independent, but together they support the twofold contribution of the thesis.

For the ecological half, the evaluation is centred on explanatory fit over the 2005–2021 window, using scenario-based counterfactual analysis rather than free forecasting. The reference against which everything is compared is the Romanian national annual series, which rises from about 5800 bears in 2005 to about 7500 bears in 2021. That implies an observed cumulative increase of roughly 29.3% over the window and an average annual growth of about 1.62% per year. These two numbers, the end level and the growth rate, are the targets every scenario is trying to approach.

The simulations are compared against the reference trajectory using four indicators, chosen so that no single number can hide a bad fit:

- the mean absolute error (MAE) between the simulated and observed annual values,
- the root mean square error (RMSE), which penalises large yearly deviations more heavily than MAE,
- the end-of-window gap to the 2021 reference level,

- the cumulative growth and the annualized growth (CAGR) inside the same window.

MAE and RMSE measure how close the whole trajectory stays to the reference; the end-of-window gap measures whether the simulation arrives at roughly the right level by 2021; and the growth figures measure whether it got there by growing at a believable rate rather than by some accident of the starting point. Reporting all four together is what lets us rank scenarios with some confidence instead of cherry-picking the metric that flatters a given run.

4.2 Experimental protocol and scenarios

Every scenario was run with three replicates, each with its own deterministic seed, and the per-tick output was then post-processed into annual aggregates, mapping simulation year 0 to 2005 and year 16 to 2021. Three replicates is modest, but because each run is itself an average over a thousand-bear population the between-replicate variance is small, and the aggregation smooths the hourly tick output into the yearly resolution at which the reference data actually exists. The campaign covers a baseline plus six driver scenarios:

- baseline, with no time-varying driver beyond the calibrated default settings,
- the 2007 EU-protection driver, lowering danger-related mortality from year 2,
- the 2016 hunting-ban driver, lowering danger mortality from year 11,
- the 2021 intervention-cull driver, raising danger mortality from year 16,
- a habitat-loss driver, reducing forest food in steps,
- an anthropogenic-food driver, raising food regrowth in steps,
- a combined realistic schedule that layers protection, food increase, the 2016 ban and the 2021 cull together.

Each of these is expressed purely as a parameter schedule applied through the *ScheduleApplier* described in Chapter 3; no scenario required any change to the simulation loop. Table 4.1 summarises the schedule effect behind each one.

It is worth noting that the scenarios start from a spun-up population rather than from raw random founders, using the snapshot and balanced-initialization machinery from Chapter 3. This matters for the fairness of the comparison: because every scenario begins from the same realistic demographic and spatial structure, the differences we measure between them can be attributed to the drivers themselves and

Table 4.1: Scenario definitions used in the final campaign.

Scenario	Main schedule effect
Baseline	No time-varying policy/habitat driver beyond baseline settings.
EU protection 2007	Danger mortality reduced from simulation year 2.
Hunting ban 2016	Danger mortality reduced from simulation year 11.
Intervention cull 2021	Danger mortality increased from simulation year 16.
Habitat loss	Forest food reduced stepwise (years 3, 9, 14).
Anthropogenic food	Food regrowth increased stepwise (years 5, 10, 15).
Combined realistic	Layered schedule: protection + food increase + 2016 ban + 2021 cull.

not to leftover initialization wobble. The first few simulated years therefore already sit at a sensible level around 5870–5940 bears, close to the 2005 reference, instead of crashing and rebounding the way a freshly seeded uniform founding cohort would.

4.3 Ecological results

4.3.1 Quantitative summary

Table 4.2 shows the core quantitative comparison of every scenario against the 2005–2021 reference trajectory.

Table 4.2: Ecological validation metrics (annual aggregates, 2005–2021).

Scenario	End pop.	Growth (%)	CAGR (%/y)	MAE	RMSE
Baseline	6020.67	2.57	0.16	360.32	539.66
EU protection 2007	6103.33	3.98	0.24	342.60	511.44
Hunting ban 2016	6068.67	3.39	0.21	348.76	522.58
Intervention cull 2021 (16y)	6020.67	2.57	0.16	360.32	539.66
Habitat loss	6020.67	2.57	0.16	360.32	539.66
Anthropogenic food	7175.00	22.24	1.26	261.25	336.27
Combined realistic	7194.67	22.58	1.28	278.64	344.65
Reference (observed)	7500.00	29.31	1.62	0.00	0.00

Reading down the table, a fairly clear story emerges. The baseline and the policy-only schedules all stay close together near the bottom: they grow only a couple of percent over the whole window and finish around 6000–6100 bears, well short of the 7500 reference. The danger-mortality drivers (EU protection 2007 and the 2016 hunting ban) do nudge the fit in the right direction (MAE falls from 360 to about

343 and 349 respectively), but the improvement is small. This is because lowering danger mortality removes only a modest slice of total deaths, and, as we will see in a moment, danger is not where most of the dying actually happens in this model.

The two food-driven schedules are in a completely different regime. The anthropogenic-food scenario reaches 7175 bears by 2021, a 22% increase and a CAGR of 1.26% per year, and it gives the best fit in the whole campaign with an MAE of 261.25 and an RMSE of 336.27. The combined realistic schedule is almost identical in trajectory (7194.67 bears, MAE 278.64, RMSE 344.65), but it does not beat anthropogenic food on aggregate error. Relative to baseline, the MAE drops by about 27.5% under anthropogenic food and by about 22.7% under the combined schedule. The fact that the combined schedule, which contains the food increase *plus* the policy drivers, does not improve on the food driver alone is itself informative: in this calibration the extra policy components either contribute little or partially cancel one another, and the food pathway is doing essentially all of the explanatory work.

4.3.2 Where the trajectories diverge

The aggregate metrics hide *when* the scenarios part ways, which is probably the most interesting part of the result. Table 4.3 lines up the annual population for the baseline against the anthropogenic-food scenario, year by year.

Table 4.3: Annual mean population, baseline versus anthropogenic-food scenario (three-replicate aggregates).

Sim. year (calendar)	Baseline	Anthropogenic food	Difference
0 (2005)	5869.60	5869.60	0.00
2 (2007)	5938.59	5938.59	0.00
4 (2009)	5938.91	5938.91	0.00
5 (2010)	5934.91	5933.70	-1.21
6 (2011)	5999.94	5987.34	-12.60
8 (2013)	6074.16	6136.15	+61.99
10 (2015)	6009.72	6268.89	+259.17
12 (2017)	6039.79	6601.82	+562.03
14 (2019)	6005.54	6946.73	+941.19
16 (2021)	6020.67	7175.00	+1154.33

Up to year 5 the two trajectories are identical, which is expected, because the food schedule only begins acting in year 5. For a year or two after that the food-driven run is even marginally *below* baseline, since the first stepwise food increase takes time to translate into higher satiety, then into more females clearing the reproduction threshold, then into surviving cubs. From around year 8 the curves separate for good, and the gap widens every year, reaching over 1150 bears by 2021. This lag-then-acceleration shape is biologically sensible: more food does not produce more

bears instantly: it produces them through the slow chain of better condition, more births and better cub survival, and that chain has the inertia of the species' long reproductive cycle built into it. The baseline, by contrast, just oscillates around 6000 for the whole window, which is the signature of a population sitting at the carrying capacity that the unchanged food supply allows.

4.3.3 Mortality structure

To understand *why* the food driver works and the policy drivers barely move the needle, it helps to look at what bears actually die of. Averaged over the window, the baseline produces on the order of 510 deaths per year, and the overwhelming majority of those, around 400 per year, are from starvation. Danger-related deaths are far smaller, on the order of 35 per year, and old-age deaths sit around 70 per year. Births average somewhere between 500 and 580 per year, which is why the baseline population stays roughly flat: births and deaths are close to balanced.

This composition explains the whole ranking. The policy drivers act only on danger mortality, which is a small component of total deaths, so even halving it can only recover a few dozen bears a year. The food driver, on the other hand, acts directly on the dominant cause of death: more food means fewer bears starve, and at the same time more females meet the satiety requirement to reproduce. It pushes on the largest lever in the system, on both the mortality side and the birth side at once, which is exactly why it is the only mechanism in the campaign that closes most of the gap to the observed trajectory. Across the best-fit scenarios the mean annual total deaths fall from about 513 in the baseline to around 491, and the share of danger deaths drops slightly in the combined schedule, consistent with its lower danger-mortality setting over part of the timeline.

4.3.4 End-of-window interpretation

At year 16, mapped to 2021, the reference value is 7500 bears. The baseline ends 1479 bears short of that; the anthropogenic-food scenario ends only 325 short, and the combined realistic scenario only 305 short. So while none of the scenarios reaches the full observed level, the food-driven schedules close roughly four-fifths of the gap that the baseline leaves open. In the current parameterization, then, increasing food availability over time is the mechanism that comes closest to reproducing the observed 2021 level, and it is hard to reach that level at all without it.

4.4 Trajectory and structure figures

The figures in this section are produced from the per-tick CSV output, aggregated to annual values and averaged over the three replicates of each schedule. Figure 4.1 collects the mean population trajectories of all seven schedules over the 2005–2021 window (simulation years 0 to 16). The contrast described above is visible at a glance: the baseline and the policy-only schedules stay flat around 6000 bears and even dip near year 15, while the anthropogenic-food and combined-realistic schedules climb steadily from about year 8 onward.

Figures 4.2 and 4.3 then turn to the demographic structure behind those trajectories. Figure 4.2 breaks the baseline down by cause of death, and it makes the central mortality argument concrete: starvation is by far the dominant cause, on the order of several hundred deaths per year, while danger and old-age mortality are an order of magnitude smaller. This is exactly why a driver that adds food moves the population so much more than one that only lowers danger mortality. Figure 4.3 compares births against total deaths for the baseline and for the best-fit anthropogenic-food schedule, showing how the food driver tips the births-deaths balance toward sustained growth.

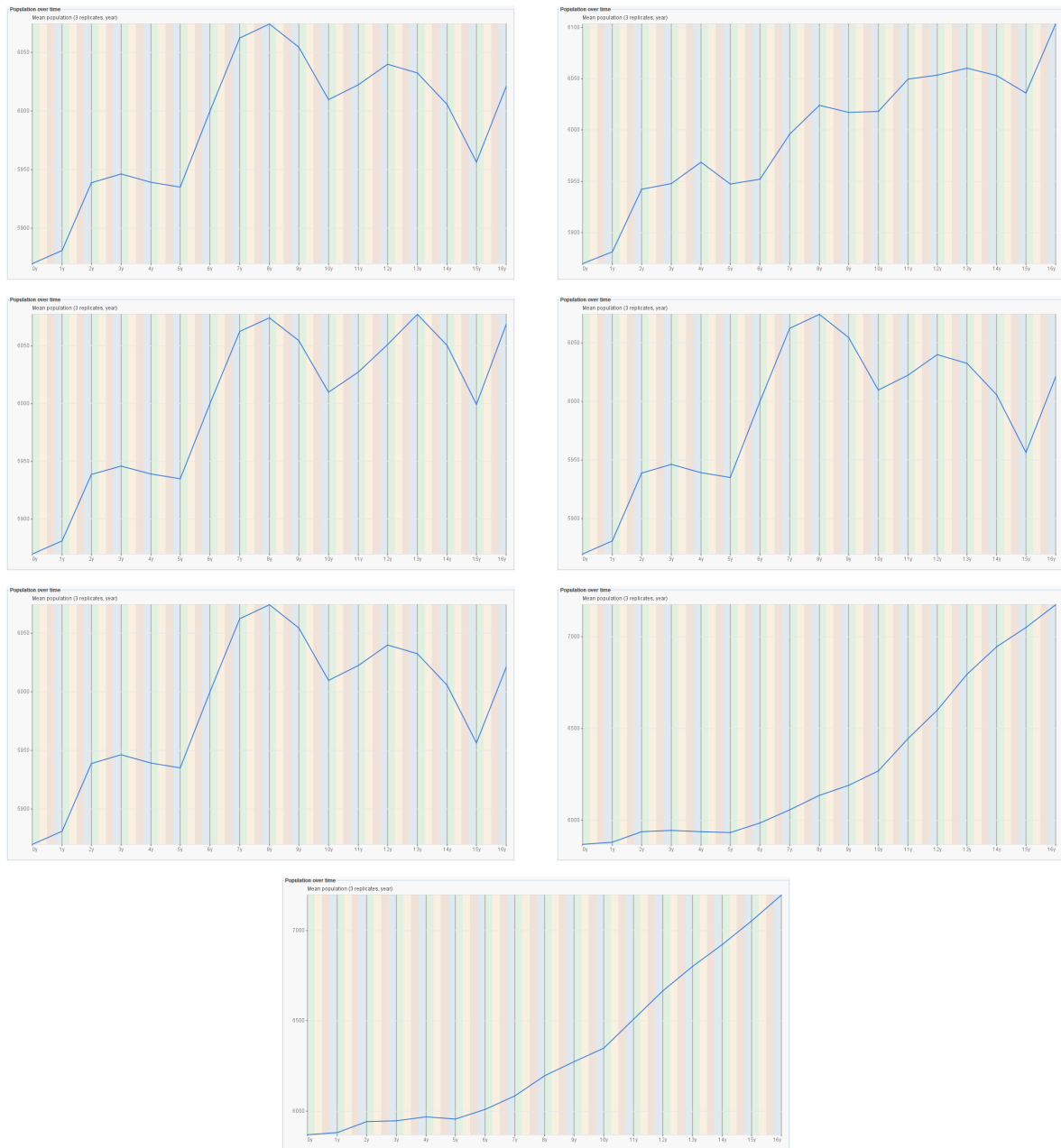


Figure 4.1: Mean simulated population over time (three replicates, simulation years 0–16, i.e. 2005–2021), one panel per schedule. Reading left to right, top to bottom: baseline; EU protection 2007; hunting ban 2016; intervention cull 2021; habitat loss; anthropogenic food; combined realistic. The two food-driven schedules (anthropogenic food and combined realistic) are the only ones that produce a sustained rise, while the baseline and the policy-only schedules stay near 6000 bears across the window.



Figure 4.2: Baseline mortality structure (mean per year, three replicates). Top left: total deaths; top right: starvation deaths; bottom left: danger deaths; bottom right: old-age deaths. Starvation accounts for the large majority of mortality, which is the reason food availability is the most effective lever in the campaign.



Figure 4.3: Births versus total deaths over time (mean per year, three replicates). Top row: baseline births (left) and baseline total deaths (right). Bottom row: anthropogenic-food births (left) and anthropogenic-food total deaths (right). Under the food driver, births stay high while deaths fall, and it is this widening gap between births and deaths that produces the sustained population increase seen in Figure 4.1.

4.5 Computational analysis

The second half of the evaluation concerns the execution strategy. The position taken in Chapter 3 was that multithreading is the right tool for accelerating one run, and that distributing a single run would not be worthwhile for a model of this granularity, which is why a distributed mode was never implemented. The timing instrumentation built into the simulation is what lets us support that position with measurements rather than just assert it. To be explicit about what is and is not measured here: the figures below come from the multithreaded, single-machine execution that the project actually uses; there are no distributed-execution measurements, because there is no distributed execution. What the instrumentation gives us instead is the per-tick anatomy that explains *why* distributing a single run would not have helped.

4.5.1 Where a tick spends its time

When benchmarking is enabled, every tick is decomposed into a planning phase and a commit phase, and each of those is broken down further, as described in Chapter 3. The planning phase covers building one task per agent, invoking them all on the virtual-thread executor, and collecting their results, together with the aggregate worker time spent on percept generation and on decision-making. The commit phase covers contributing the actions into the effects container, applying deaths, applying births, applying food reductions, resolving moves, recording conflict exposure, regrowing food and updating counters. The instrumentation also tracks an explicit “unaccounted” remainder in each phase, so the breakdown adds up honestly.

The qualitative picture that this produces, and the one that matters for the architecture argument, is the following. The planning phase is the part that actually parallelises: it is made of one independent, side-effect-free task per bear, and with virtual threads we can run thousands of them concurrently on a single machine, so on multi-core hardware the wall-clock cost of planning grows much more slowly than the number of bears. The commit phase, by contrast, is strictly serial, it runs on one driver thread because that is what guarantees the deterministic, race-free world transition, and its cost grows roughly linearly with the number of intents to resolve. Each individual commit operation is cheap, but the whole commit is an unavoidable synchronisation barrier that closes every single tick.

4.5.2 Why a single run should not be distributed

This decomposition is exactly what makes distributing a single run unattractive, and the argument is now concrete rather than hand-waved. The useful work that distribution would parallelise, the per-agent planning, is on the order of microseconds per bear. But every tick ends in a barrier, and a run over the sixteen-year window is on the order of 140,000 ticks long. To distribute one run across machines, each of those 140,000 ticks would require gathering every machine's intents to a coordinator, resolving them serially, and broadcasting the updated world back before the next tick could begin. The communication and synchronisation cost of crossing the network barrier 140,000 times completely dominates the tiny per-agent work it would be spreading out. In other words, the model is fine-grained and tightly synchronised, and that is precisely the workload shape that the parallel-and-distributed-simulation literature identifies as a poor fit for distribution [Fuj00]. Multithreading wins here because the threads share memory and the commit barrier costs essentially nothing; a network barrier of the same frequency would not.

4.5.3 Where distribution would have belonged, and why it was still not needed

The flip side is that the campaign as a whole would have been an almost ideal distributed workload, had distribution been required at all. It consists of many runs, seven scenarios times three replicate seeds, and in practice far more during calibration, and these runs share nothing. They do not communicate, they never synchronise with each other, and each one produces its own independent CSV output. This is the textbook “embarrassingly parallel” situation: each run could in principle be handed to a separate machine and left alone until it finishes, and the *BatchRunner* already treats runs as independent units, which is exactly the structure one would distribute. So if any part of this project were ever to be distributed, it is the batch and not the single run.

In practice, though, it was not necessary. The *BatchRunner* executes the scenarios sequentially on one multi-core machine, where the multithreaded planning already keeps each individual run fast, and the full campaign finishes in an acceptable amount of time. Adding a real distributed batch layer, a job scheduler, result collection across nodes, the associated failure handling, would have been pure engineering overhead with no payoff at this scale, so it was deliberately left out. The conclusion of the computational analysis is therefore a clean, slightly contrarian one: multithreading belongs inside the run and is used; distribution would only belong around the batch and was not used, because a single machine sufficed.

4.6 Interpretation and validity limits

The strongest and most defensible conclusion from this campaign is comparative rather than absolute: in the schedules tested and under the current calibration, an increase in food access explains substantially more of the 2005–2021 rise than policy-only shifts in danger mortality do. That conclusion follows cleanly from the metrics, the year-by-year divergence, and the mortality structure, which all point the same way.

Two limits should be stated plainly so the claim is not overread. First, two of the result folders, the anthropogenic-food and combined-realistic outputs, are marked in the data directory as needing a reproducibility recheck before they are used for publication-grade attribution, because there is a chance of pipeline contamination between those two closely related scenarios; the comparative ranking is robust, but the precise error figures for those two should be re-confirmed under a strict protocol. Second, the 2021 intervention-cull driver is activated at simulation year 16, the very end of the comparison window, so it has almost no time to act inside 2005–2021 and is essentially identical to baseline there. That near-identity must be read as a consequence of timing, not as evidence that culling has no effect; quantifying its real impact requires extending the horizon past 2021. Even with these two caveats, the campaign is strong enough to support a thesis-level conclusion about which mechanisms are necessary to approach the observed growth, and the computational half of the evaluation independently supports the execution strategy that produced these results.

Chapter 5

Discussion, conclusion and future work

5.1 Discussion

Taken together, the results of the final campaign give a fairly clean ranking of explanatory power inside the 2005–2021 validation window, and it is a ranking that I find convincing precisely because the different pieces of evidence all agree with one another instead of pulling in different directions.

The baseline and the policy-only schedules, the 2007 EU protection and the 2016 hunting-ban reductions in danger mortality, improve the fit only marginally relative to the baseline. The schedules that increase food access over time, by contrast, produce a clearly stronger match to the observed trajectory. The single best scenario in the run set is anthropogenic food, with an MAE of 261.25 against 360.32 in the baseline, an RMSE of 336.27 against 539.66, and an end population of 7175 against the 7500 reference for 2021. The combined realistic schedule comes very close (MAE 278.64, RMSE 344.65, end population 7194.67) but does not actually beat anthropogenic food on aggregate error. That last point is more interesting than it first looks: the combined schedule contains the food increase *and* the policy drivers, so the fact that adding the policies on top of the food does not improve the fit tells us that, in this parameterization, the food pathway is doing essentially all of the explanatory work, while the additional policy components contribute little or partly offset one another.

The reason behind this ranking becomes obvious once we look at the mortality structure. In the model, the overwhelming majority of deaths are from starvation, on the order of four hundred a year against only a few dozen from danger. The policy drivers act solely on danger mortality, which is the small component, so even a large relative cut to it can only recover a handful of bears. The food driver acts on the

dominant component directly: more food means fewer bears starve, and at the same time more females clear the satiety requirement and reproduce. It pushes on the biggest lever in the system from both the death side and the birth side at once, which is exactly why it is the only mechanism in the campaign able to close most of the gap to the observed level. The year-by-year trajectory reinforces this with a believable lag: the food increase begins in year 5 but the curves do not visibly separate until around year 8, because more food converts into more bears only through the slow chain of better body condition, then more births, then surviving cubs, a chain that carries the long reproductive cycle of the species inside it.

A second point worth dwelling on is horizon sensitivity, because it is easy to misread. The 2021 intervention-cull schedule is activated at simulation year 16, the very end of the comparison window, so within 2005–2021 it has essentially no time to act and comes out nearly identical to the baseline. This should not be read as “culling does nothing”. It is purely a consequence of timing: the driver fires too late to express itself inside the validated window, and measuring its real effect would require extending the simulation past 2021. This is a good illustration of why a schedule-driven agent-based model is valuable for this kind of policy question in the first place: it lets us reason not only about whether a driver causes growth or decline, but about *when* a driver is applied and how long it needs to take effect.

5.2 Conclusion

This thesis set out to answer which mechanisms can best explain the increase of the Romanian brown bear population over 2005–2021, and to do so with a model that is reproducible and whose computational behaviour is understood rather than assumed. It delivers both a methodological contribution and a data-backed empirical conclusion.

Methodological contribution

1. A spatial, agent-based brown bear simulator with explicit demographic and environmental mechanisms, age- and sex-structured mortality, satiety-gated reproduction with gestation and cooldown, variable litters with infant mortality, seasonal hibernation, subadult-male dispersal, and a food/danger landscape that the bears both consume and respond to.
2. A reproducible scenario pipeline built directly into the code: per-agent seeded randomness that survives parallel execution, time-indexed parameter schedules applied and audited through reflection, seeded replicates, spin-up via

snapshots and balanced initialization, annual post-processing, and quantitative fit metrics.

3. A practical and somewhat unusual computational result for this class of model: the model is multithreaded on a single machine, and a distributed execution was deliberately not implemented, because the timing analysis shows that a fine-grained, tick-synchronised run would lose more to communication and synchronisation overhead than it could gain. Distribution would only make sense around a batch of independent runs, and at this scale a single machine made even that unnecessary.

Empirical conclusion for 2005–2021

Within the tested scenarios and the current calibration, population growth close to the observed Romanian trajectory requires an increase in food availability over time. Policy-only adjustments to danger mortality are not, by themselves, sufficient to reproduce the magnitude of the rise. In short, the campaign supports a multi-driver interpretation in which food-access dynamics are both necessary and dominant in this version of the model. This is a comparative statement, made carefully and within stated assumptions, and it is exactly the kind of statement the scope of the thesis was designed to be able to support.

5.3 Limitations

I would rather state the limitations plainly than have them found, because each of them marks the boundary of where the present work applies, not a flaw that undermines it.

1. Two of the result folders, the anthropogenic-food and the combined-realistic outputs, are flagged in the data directory as needing a reproducibility recheck before they are used for publication-grade attribution, because of a possible pipeline contamination between two closely related scenarios. The comparative ranking is robust, but the precise error figures for those two scenarios should be re-confirmed under a strict, isolated protocol.
2. The validation uses a single national annual reference trajectory. Spatially explicit, local validation, checking that the model gets the right bears in roughly the right places, is outside the current scope, and a good fit to the national curve does not by itself guarantee a correct spatial pattern.

3. Some drivers, especially the late-window 2021 cull, are activated too near the end of the window to express their full effect, so their real impact can only be quantified with an extended horizon.
4. Several ecological processes are intentionally abstracted: food is a single scalar field rather than distinct food types with their own seasons, human–bear conflict is a proxy count of bears on village and road cells rather than a modelled human response, and there is no individual-quality or genetic variation between bears beyond age and sex.

These limitations constrain the strength of the claims, but they do not invalidate the comparative ranking that the campaign produced, and most of them point directly at the next steps.

5.4 Future work

There are five directions in which I would take this work further, and they follow naturally from the limitations above.

1. Run a post-2021 extension campaign, for example to 2026 or 2030, specifically to measure the delayed effect of the intervention-cull driver and of any other late-window policy, which the current window cannot capture.
2. Recheck and rerun the anthropogenic-food and combined scenarios under a strict, fully isolated reproducibility protocol, to rule out the possible pipeline contamination and to confirm the precise error figures.
3. Add uncertainty envelopes, confidence bands across replicates, directly onto the final trajectory figures, so that the comparison between scenarios is shown with its variability rather than as bare lines.
4. Extend the validation with additional anchors, for example independent genetic population estimates, while keeping the annual-series evaluation separate, so that the model is checked against more than one kind of data.
5. Add a dedicated benchmark appendix tied to one final, frozen code-and-data snapshot, so that the ecological and computational conclusions are both anchored to the exact same version of the system, and so that the multithreaded scaling (wall-clock time against agent count and core count) can be reported as hard numbers on specific hardware. If the experiment scale ever grew enough to justify it, the same appendix would be the place to prototype and measure a distributed batch layer and confirm in practice the cost-benefit argument made here.

Beyond these, the natural longer-term extensions are to enrich the environment, distinct food types with their own seasonality, a real loaded habitat grid for Romania's forest range instead of a statistical terrain mix, and to model the human side of conflict explicitly rather than as a proxy count.

5.5 Final remarks

The thesis is in a complete state: the numerical evidence is integrated, the scenario ranking is explicit, and the result figures are in place. What remains before submission is mechanical rather than conceptual, that is cross-checking every table value against the final exported CSVs and a last pass over the narrative for consistency of captions, references and the links between the schedules and their results. The substance of the argument, ecological and computational alike, is already on the page.

Bibliography

- [BPP⁺14] Greta Bocedi, Stuart C. F. Palmer, Guy Pe'er, Risto K. Heikkinen, Yian-nis G. Matsinos, Kevin Watts, and Justin M. J. Travis. Rangesifter: A platform for modelling spatial eco-evolutionary dynamics and species responses to environmental changes. *Methods in Ecology and Evolution*, 5(4):388–396, 2014.
- [BSY⁺09] Richard Bischof, Jon E. Swenson, Nigel G. Yoccoz, Atle Mysterud, and Olivier Gimenez. The magnitude and selectivity of natural and multiple anthropogenic mortality causes in hunted brown bears. *Journal of Animal Ecology*, 78(3):656–665, 2009.
- [CKL⁺14] Guillaume Chapron, Petra Kaczensky, John D. C. Linnell, Luigi Boitani, et al. Recovery of large carnivores in europe’s modern human-dominated landscapes. *Science*, 346(6216):1517–1519, 2014.
- [CLBG15] Neil H. Carter, Simon Levin, Andrew Barlow, and Volker Grimm. Modeling tiger population and territory dynamics using an agent-based approach. *Ecological Modelling*, 312:347–362, 2015.
- [CN13] Nicholson Collier and Michael North. Parallel agent-based simulation with repast for high performance computing. *Simulation*, 89(10):1215–1235, 2013.
- [DM05] Donald L. DeAngelis and Wolf M. Mooij. Individual-based modeling of ecological and evolutionary processes. *Annual Review of Ecology, Evolution, and Systematics*, 36:147–168, 2005.
- [Fuj00] Richard M. Fujimoto. *Parallel and Distributed Simulation Systems*. Wiley, New York, 2000.
- [GBB⁺06] Volker Grimm, Uta Berger, Finn Bastiansen, Sondre Eliassen, Vincent Ginot, Jarl Giske, John Goss-Custard, Tamara Grand, Simone Heinz, Geir Huse, Andreas Huth, Jane U. Jepsen, Christian Jorgensen, Wolf M.

- Mooij, Birgit Muller, Guy Pe'er, Cyril Piou, Steven F. Railsback, Andrew M. Robbins, M. Martha Robbins, Eva Rossmanith, Nadja Ruger, Espen Strand, Sami Souissi, Richard A. Stillman, Rune Vabo, Ulf Visser, and Donald L. DeAngelis. A standard protocol for describing individual-based and agent-based models. *Ecological Modelling*, 198(1–2):115–126, 2006.
- [GMKH13] Patrick Griffiths, Daniel Muller, Tobias Kuemmerle, and Patrick Hostert. Agricultural land change in the carpathian ecoregion after the breakdown of socialism and expansion of the european union. *Environmental Research Letters*, 8(4):045024, 2013.
- [GR05] Volker Grimm and Steven F. Railsback. *Individual-Based Modeling and Ecology*. Princeton University Press, Princeton, 2005.
- [GRV⁺20] Volker Grimm, Steven F. Railsback, Christian E. Vincenot, Uta Berger, Cara Gallagher, Donald L. DeAngelis, Bruce Edmonds, Jiaqi Ge, Jarl Giske, Jurgen Groeneveld, Alison S. A. Johnston, Andrew Milles, Jacob Nabe-Nielsen, J. Gareth Polhill, Viktoriia Radchuk, Marie-Sophie Rohwader, Richard A. Stillman, Jan C. Thiele, Daniel Ayllon, Els Augustijn-Beckers, Matthias Becher, Vincent Bretagnolle, Jan Harke, Carol R. Johnston, Benjamin T. Martin, James Mollison, Birgit Muller, Sarah Ogilvy, Fabien Quetier, Amelie Schmolke, Pernille Thorbek, and Damaris Zurell. The odd protocol for describing agent-based and other simulation models: A second update to improve clarity, replication, and structural realism. *Journal of Artificial Societies and Social Simulation*, 23(2):7, 2020.
- [INC23] INCDS Marin Dracea Brasov. National brown bear genetic census report. Institutional technical report, 2023. Romania.
- [LCRP⁺05] Sean Luke, Claudio Cioffi-Revilla, Liviu Panait, Keith Sullivan, and Gabriel Balan. Mason: A multiagent simulation environment. *Simulation*, 81(7):517–527, 2005.
- [Mac16] Charles M. Macal. Everything you need to know about agent-based modelling and simulation. *Journal of Simulation*, 10(2):144–156, 2016.
- [McL89] Bruce N. McLellan. Dynamics of a grizzly bear population during a period of industrial resource extraction. i. density and age-sex composition. *Canadian Journal of Zoology*, 67(8):1856–1860, 1989.

- [Min24] Ministerul Mediului, Apelor si Padurilor. Planul national de actiune pentru conservarea populatiei de urs brun din romania. Government policy document, 2024. Romania.
- [MK15] David Masad and Jacqueline Kazil. Mesa: An agent-based modeling framework. In *Proceedings of the 14th Python in Science Conference*, pages 53–60. SciPy, 2015.
- [MKB⁺14] Cristina Munteanu, Tobias Kuemmerle, Miroslav Boltiziar, Van Butsic, et al. Forest and agricultural land change in the carpathian region: A meta-analysis of long-term patterns and drivers of change. *Land Use Policy*, 38:685–697, 2014.
- [NCO⁺13] Michael J. North, Nicholson T. Collier, Jonathan Ozik, Eric R. Tatara, Charles M. Macal, Mark Bragen, and Pamela Sydelko. Complex adaptive systems modeling with repast symphony. *Complex Adaptive Systems Modeling*, 1:3, 2013.
- [PBC⁺18] Maria I. Pop, Lia Bereczky, Sorin Chiriac, Razvan Iosif, Andra Nita, Viorel D. Popescu, and Laurentiu Rozylowicz. Movement ecology of brown bears (*ursus arctos*) in the romanian eastern carpathians. *Nature Conservation*, 26:15–31, 2018.
- [PIP⁺17] Viorel D. Popescu, Razvan Iosif, Maria I. Pop, Sorin Chiriac, George Buros, and Brian J. Furnas. Integrating sign surveys and telemetry data for estimating brown bear (*ursus arctos*) density in the romanian carpathians. *Ecology and Evolution*, 7(18):7134–7144, 2017.
- [RG19] Steven F. Railsback and Volker Grimm. *Agent-Based and Individual-Based Modeling: A Practical Introduction*. Princeton University Press, Princeton, 2nd edition, 2019.
- [SEH⁺12] Sam M. J. G. Steyaert, Anders Endrestol, Klaus Hacklander, Jon E. Swenson, and Andreas Zedrosser. The mating system of the brown bear *ursus arctos*. *Mammal Review*, 42(1):12–34, 2012.
- [SES⁺98] Bernt-Erik Saether, Steinar Engen, Jon E. Swenson, Oivind Bakke, and Fredrik Sandegren. Assessing the viability of scandinavian brown bear, *ursus arctos*, populations: the effects of uncertain parameter estimates. *Oikos*, 83:403–416, 1998.
- [SGDZ00] Jon E. Swenson, Norbert Gerstl, Bjorn Dahle, and Andreas Zedrosser. Action plan for the conservation of the brown bear in europe (*ursus arctos*). Technical report, Council of Europe, Strasbourg, 2000.

- [SMH03] Charles C. Schwartz, Sterling D. Miller, and Mark A. Haroldson. Grizzly bear. In *Wild Mammals of North America: Biology, Management, and Conservation*, pages 556–586. Johns Hopkins University Press, Baltimore, 2nd edition, 2003.
- [SRA⁺08] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. Wiley, Chichester, 2008.
- [SSS⁺97] Jon E. Swenson, Fredrik Sandegren, Andreas Soderberg, Anders Bjarrvall, Roger Franzen, and Petter Wabakken. Infanticide caused by hunting of male bears. *Nature*, 386:450–451, 1997.
- [TKG14] Jan C. Thiele, Winfried Kurth, and Volker Grimm. Facilitating parameter estimation and sensitivity analysis of agent-based models: A cookbook using netlogo and r. *Journal of Artificial Societies and Social Simulation*, 17(3):11, 2014.
- [WR15] Uri Wilensky and William Rand. *An Introduction to Agent-Based Modeling*. MIT Press, Cambridge, MA, 2015.